



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

开源软件开发实践实验手册

Lab4

开源软件开发中的 DevOps



计算学部

哈尔滨工业大学

2024 年秋季学期

目录

1	实验目标.....	1
2	实验内容.....	1
2.1	Github Actions DevOps 实践.....	1
2.2	Jenkins DevOps 实践.....	5
3	实验报告.....	11

1 实验目标

本次实验训练开源软件开发中的基本 DevOps 操作，具体来说：

- 掌握开源软件开发中的基本 DevOps 流程和工具的使用
- 熟悉利用 Github Actions 进行 DevOps
- 熟悉利用 Jenkins 进行 DevOps

2 实验内容

本次实验在 GitHub Classroom 中的 URL 地址为：
<https://classroom.github.com/a/T75S58J->。请访问该 URL，按照提示建立自己的 Lab4 仓库并关联至自己的学号。仓库命名规则为：Lab4-学号。

请参照下文的实验步骤和指导，完成 2 个实验。

2.1 Github Actions DevOps 实践

Github Actions 详细文档见：<https://docs.github.com/zh/actions>

(一) 实验开发环境

- Java JDK 1.8 以上（建议使用 JDK 11+）
- Git
- Maven

Java JDK、Git、Maven 安装与设置，自行上网查询学习。

(二) 测试用 github 仓库

利用 classroom 创建的实验仓库完成本次实验，个别情况下，如果遇到 Github 权限问题，可以将仓库权限修改为 public。

(三) 本地创建 maven 项目

在本地新建一个maven项目，并引入junit依赖。

junit-maven依赖如下：

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.1</version>
```

```
<scope>test</scope>
</dependency>
```

将 Lab2 中的 Debug 的程序（修改正确后的）和编写的单元测试文件，导入到项目中。

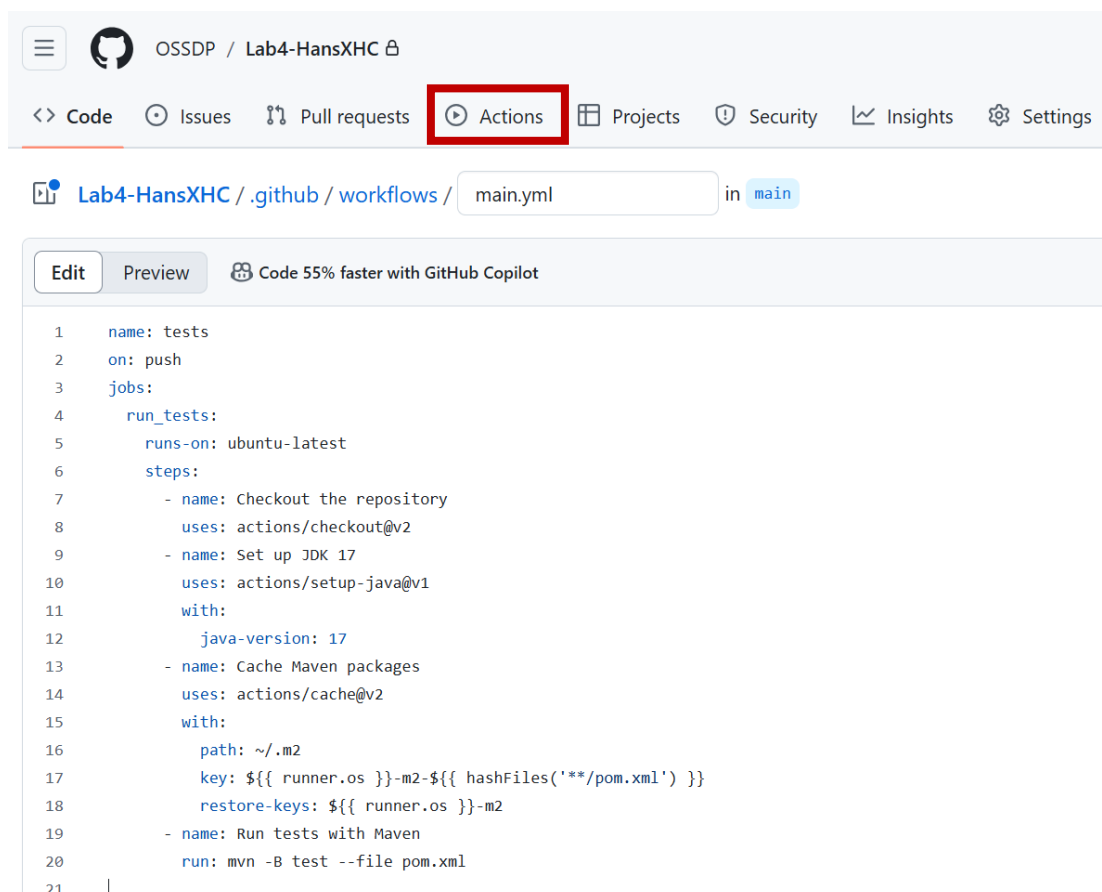
(四) 提交 maven 项目到步骤二中仓库

(五) 编写 DevOps workflow 文件

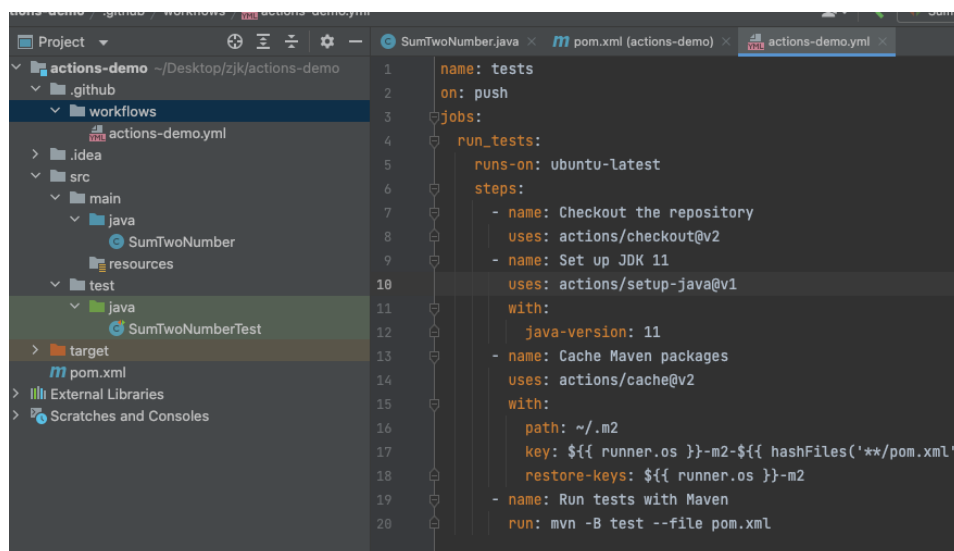
在项目根目录下新建文件夹.github/workflows/，然后新建***.yml文件（文件名任意），yml文件内容示例如下：

```
name: tests
on: push
jobs:
  run_tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout the repository
        uses: actions/checkout@v2
      - name: Set up JDK 17
        uses: actions/setup-java@v1
        with:
          java-version: 17//此处的JDK版本要同maven pom.xml文件中一致
      - name: Cache Maven packages
        uses: actions/cache@v2
        with:
          path: ~/.m2
          key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}
          restore-keys: ${{ runner.os }}-m2
      - name: Run tests with Maven
        run: mvn -B test --file pom.xml //这里具体到自己的pom.xml文件位置
```

这个文件的编辑也可以在Github中直接创建，如下图所示：

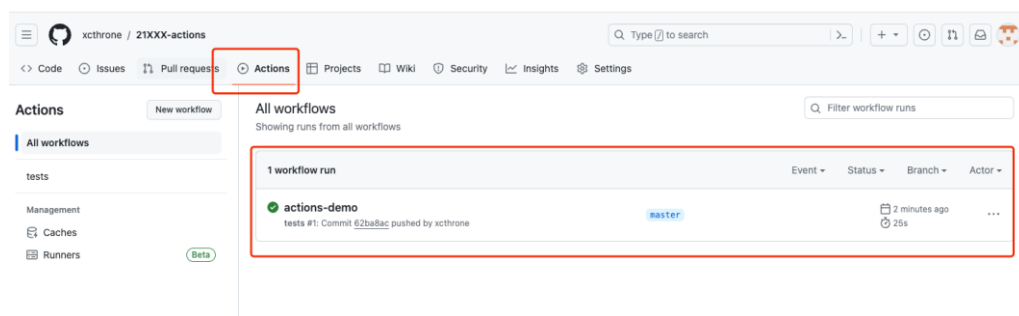


假定程序文件名字为SumTwoNumber.java，项目目录结构示意如下：

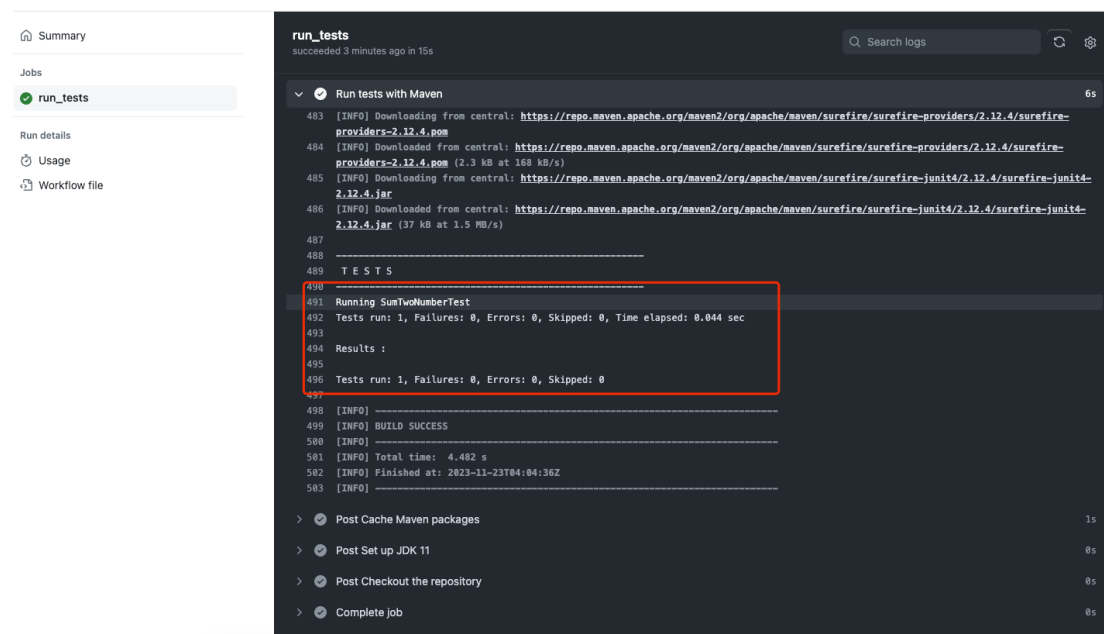


(六) 提交 workflow 文件到仓库，并执行自动化测试

编写完 workflow 文件后，再次提交到仓库，Github 会根据 yml 文件进行自动化测试。

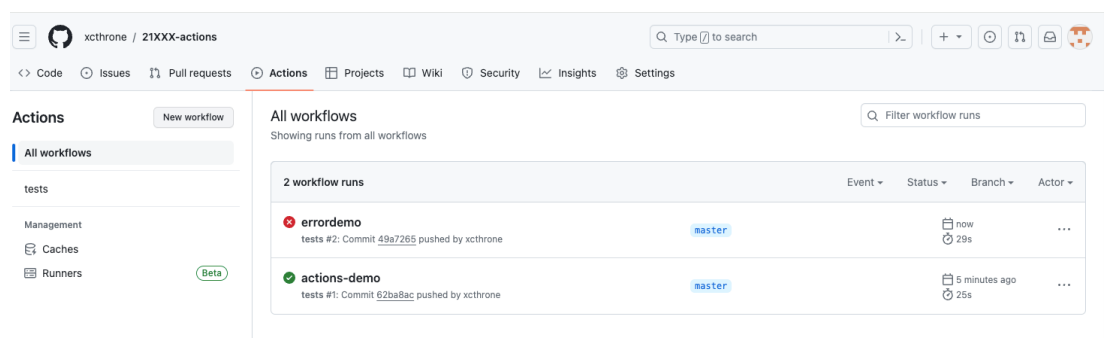


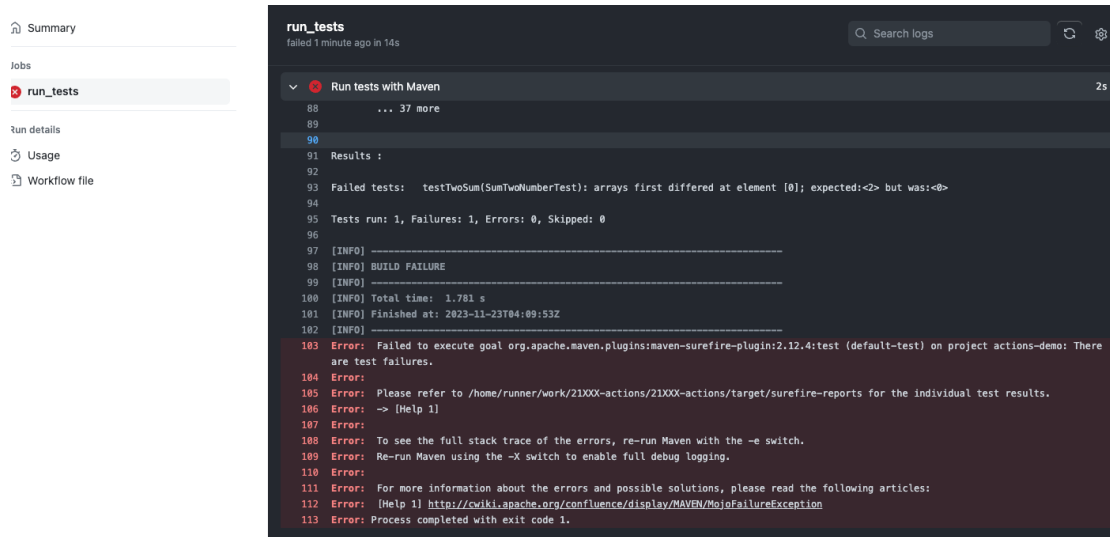
在仓库中点击Actions图标，在All workflows中可以查看到已提交的工作流。点击进去可以查看到具体的执行流程，可以看到下图中我们定义的测试用例已完成了测试。



(七) 再次执行自动化测试

修改测试用例为错误的结果，使其无法通过测试。修改后我们再次提交至仓库，下面展示了错误提交的结果。





(八) *选作任务

自行查询资料和文档，在上述 workflow 方法基础上，增加 PR 的自动 merge 功能，当某个 PR 通过了自动测试后，接受并 merge 到项目中。

2.2 Jenkins DevOps 实践

(一) 实验开发环境

- Java JDK 1.8 以上（建议使用 JDK 11+）
- Git
- Maven
- Github Client: 为支持从 cmd 进行 PR 推送，需要下载安装 Github CLI(<https://cli.github.com/>)
- Jenkins

Java JDK、Git、Maven 安装与设置，自行上网查询。

(二) Jenkins 安装与配置

在 Jenkins 的官网(<https://www.jenkins.io/zh/download/>)，根据自己计算机操作系统的类型下载安装文件，下面的操作以 Windows 版本为例。

安装时，Logon Type 选择 Run Service as Local System 以避免繁琐的额外账户配置（生产环境中应当额外配置 Jenkins 的权限水平，此实验略），Port Number 默认为 8080，输入 JAVA_HOME 地址。

安装完成后即可通过所选端口访问 Jenkins 服务，默认为 <http://localhost:8080/>。跟随系统提示，复制初始密码，解锁 Jenkins（这一步可能需要勾选“文件浏

览器”=>查看=>显示=>隐藏的项目，才能看到 ProgramData 文件夹）；选择安装推荐的插件（其中包括 Github plugin 等对实验内容有帮助的插件）；创建非管理员账号（之后的操作将在非管理员账号上执行）。

(三) Github CLI 注意事项

为了能够提交 PR，需要向 Github 认证自己，需要申请一个 Access Token。

- a) 点击 Github 头像
- b) 点击侧边栏的 settings
- c) 点击左侧最下边的 Developer settings
- d) 点击 Personal access token，选择 Token(classic)
- e) Generate new token，勾选 repo、勾选 admin:org，Generate Token 并记下 token。

(四) 测试用 github 仓库

同 2.1 使用同一个即可。

(五) 本地创建 maven 项目并提交

同 2.1 中一致。

(六) 构建 DevOps workflow

点击新建 Item 以创建新的构建流程，任务类型选择 Freestyle Project。

1. **General 部分：**该部分是对任务的描述以及一些涉及并发构建、构建结果处理、参数配置的内容，本次实验不涉及该部分内容，可自由探索。

2. **源码管理：**在该部分我们需要对前文设置的 Github 仓库进行访问配置。



此处使用 git@开头的仓库地址，以避免 HTTPS 产生的 443 超时错误。使用 SSH 访问 Github 仓库需要额外的认证凭证，点击“添加”按钮进行配置。

类型

SSH Username with private key

范围 ?

Global (Jenkins, nodes, items, all child items, etc)

ID ?

Github_Private_Key

描述 ?

Github SSH 私钥

Username

github用户名

☐ Treat username as secret ?

Private Key

☒ Enter directly

Key

添加你的私钥

一般使用 SSH Username 与私钥作为凭证，在 Username 处填入 Github 账户名，Private Key 处粘贴私钥内容（注意：粘贴时要保留第一行中的 BEGIN 和最后一行中的 END 的相关内容）。

注意：如果遇到 Unknow host 错误，这是由于 Github 服务器对于本机来说是不可信任引起的，最简单的方法即：Manage Jenkins=>Security，然后关闭 Host 检查即可。

Git Host Key Verification Configuration

Host Key Verification Strategy ?

No verification

也可以将 Github 的 host 添加到 know_host 文件中，这部分请自行探索。

指定 Local 项目代码所在的分支，本例中为 dev，实验中可根据具体情况自行设置。

Branches to build ?

指定分支 (为空白代表any) ?

/dev

Add Branch

3. 构建触发器

该部分配置如何触发构建流程。

构建触发器

☐ 触发远程构建 (例如,使用脚本) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☐ GitHub hook trigger for GITScm polling ?

☒ Poll SCM ?

日程表 ?

H/3 * * * *

Would last have run at 2023年11月23日星期四 GMT+08:00 上午12:14:56; would next run at 2023年11月23日星期四 GMT+08:00 上午

☐ 忽略钩子 post-commit ?

可以通过点击 Jenkins 页面中的 Build Now 进行构建，但为实现自动化的 CI/CD，可选择如下两种方案之一。

方案 1. 使用 GitHub hook trigger for GITScm polling，该方法由 Github 驱动，待 Github 收到 push 事件后会主动触发 Jenkins 的构建，但这依赖于公网 IP，本次实验不采用。

方案 2. 使用 Poll SCM，定时检查仓库是否有更新，进行自动化构建，按照图中配置，每 3 分钟检查一次仓库（H/3 * * * *，具体频率可自行设置，注意“*”号之间用空格分隔），本次实验选用该方案。

定时构建的语法和示例如下：

第一个*表示分钟，取值0~59

第二个*表示小时，取值0~23

第三个*表示一个月的第几天，取值1~31

第四个*表示第几月，取值1~12

第五个*表示一周中的第几天，取值0~7，其中0和7代表的都是周日

	*	*	*	*	*
含义	分钟	小时	日期	月份	星期
取值范围	0-59	0-23	1月31日	1月12日	0-7
示例					
每隔15分钟执行一次	H/15	*	*	*	*
每隔2个小时执行一次	H	H/2	*	*	*
每隔3天执行一次	H	H	H/3	*	*
每隔3天执行一次(每月的1-15号)	H	H	1-15/3	*	*
每周1,3,5执行一次	H	H	*	*	1,3,5
规则					
指定时间范围	a-b				
指定时间间隔	/				
指定变量取值	a,b,c				

每5分钟构建一次: H/5 * * * *

每15分钟运行一次: H/15 * * * *

每30分钟构建一次: H/30 * * * *

每2小时构建一次: H H/2 * * *

每天早上8点构建一次: 0 8 * * *

每天中午下班前定时构建一次: 0 12 * * *

每天下午下班前定时构建一次: 0 18 * * *

每天的8点, 12点, 22点, 一天构建3次: 0 8,12,22 * * *

一个小时的前30分钟, 每10分钟运行一次 (30分钟, 可能在 4分, 14分, 24分): H(0-29)/10 * * * *

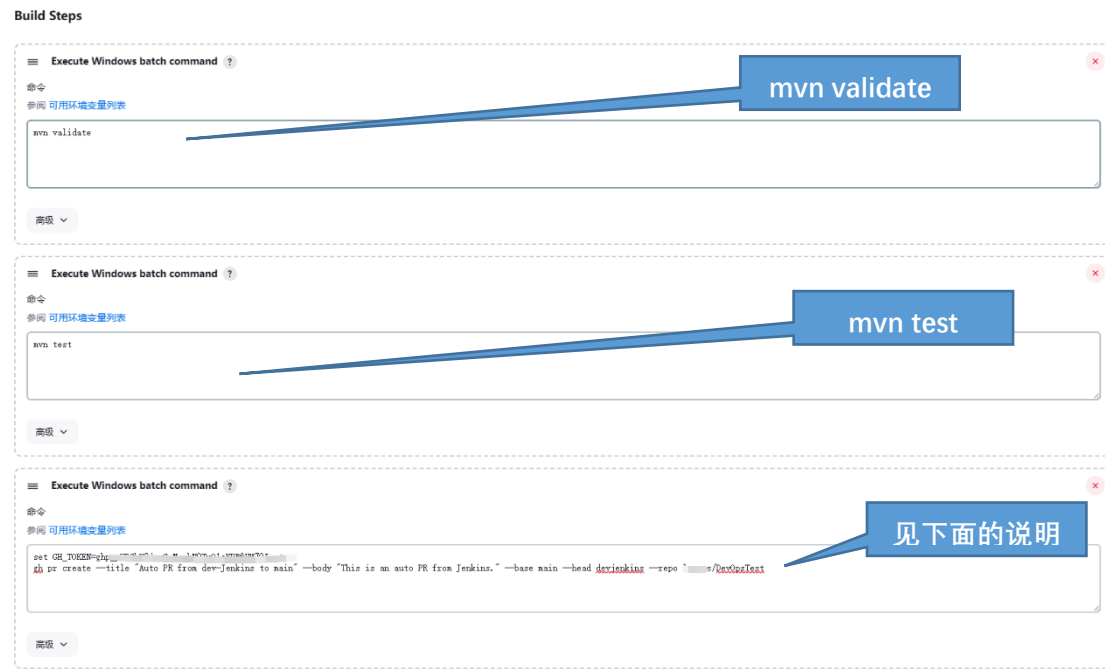
周一至周五, 上午9:45到下午3:45, 每隔2小时45分钟运行一次: 45 9-15/2 * * 1-5

每两小时一次, 每个工作日上午9点到下午5点(也许是上午10:38, 下午12:38, 下午2:38, 下午4:38): H H(9-17)/2 * * 1-5

除12月外, 每月1号和15号每天一次: H H 1,15 1-11 *

4.构建环境: 该部分用于设置构建流程的环境变量，本实验不涉及这部分内容。

5.构建步骤: 这部分将使用 cmd 脚本进行代码的测试与 PR 的提交。



set GH_TOKEN=ghp_XXXXXX // 第三步中在 Github 中生成的 token
gh pr create --title "Auto PR from dev-Jenkins to main" --body "This is an auto PR from Jenkins." --base main --head dev --repo OSSDP/Lab4-XXXX

上述命令中三个变量可以根据具体情况替换：

- main 为将要推送到的目标分支，也就是 project 的主分支
- dev 为 PR 的源分支，也就是个人写代码的分支
- OSSDP/Lab4-XXXX 为 Github 上的指定仓库，前部分为账号名称，后部为仓库名称，可根据实际情况修改

备注：如果本机没有正确配置 Maven 和 Github CLI 环境的话，执行上述命令时可能出现无法找到 mvn 或 gh 的问题，此时可使用绝对路径代替原指令中的 mvn 或 gh，如果绝对路径带有空格，使用双引号包围。

示例：C:\Users\logre\apache-maven-3.9.9\bin\mvn test

示例："C:\Program Files\GitHub CLI\gh.exe" pr create --title "Auto PR from" --body "HELLO" --base main --head dev --repo OSSDP/Lab4-XXXX

(七) 验证实验效果

将项目代码提交到选定的仓库的指定分支，本例中为 dev，等待定时任务进行构建流程。查看 Jenkins Build 的结果是否成功。

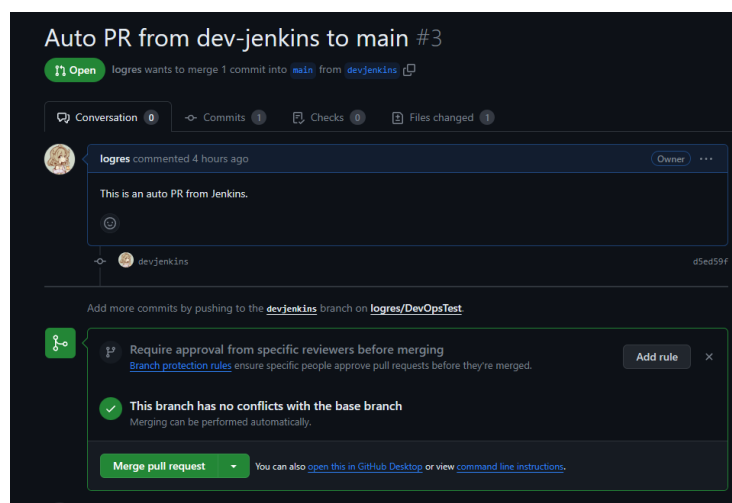
🟢 #12 (2023年11月22日 下午8:35:12)

</> No changes.

🕒 Started by an SCM change

git Revision: d5ed59f447f3b8a9ec5b3e52590633dda3af6958
Repository: git@github.com:logres/DevOpsTest.git
• origin/devjenkins

在 Github 上查看仓库的 PR 是否推送成功。



3 实验报告

针对 2 个实验内容，遵循**报告模板**，撰写简明扼要的实验报告。

- 本次实验在 GitHub Classroom 中的 URL 地址为：<https://classroom.github.com/a/T75S58J>。请访问该 URL，按照提示建立自己的 Lab4 仓库并关联至自己的学号。仓库命名规则为：Lab4-学号。注意：点击链接，系统生成个人仓库后，一定要到 github 上按照仓库命名规则修改仓库名，否则会导致批改时无法找到正确仓库。
- 实验报告请遵循实验报告模板撰写，提交到本次实验的 GitHub 仓库中；同时提交提交程序代码、YML 配置文件等。
- 提交截止日期：2024 年 12 月 10 日 23:55
- 实验报告文件可采用 Word 或 PDF 格式，命名规则：Lab4-学号-Report。
- 实验报告提交前，请在“目录”上鼠标右击，然后选择“更新域”，以确保你的目录标题/页码与正文相对应。

- 同时将实验报告、代码和配置文件打包为“Lab4-学号.zip/rar”，上传到头歌平台“Lab4 开源软件开发中的 DevOps”实验项目中。